

Advanced Technical Analysis of Whisper.cpp Server Deployment: Persistent Context, Hardware Acceleration, and Optimization Strategies on Windows Ecosystems

1. Introduction: The Paradigm Shift in Local Speech Recognition

The domain of Automatic Speech Recognition (ASR) has undergone a fundamental transformation with the release and subsequent optimization of OpenAI's Whisper model. While the original PyTorch-based implementation established new benchmarks for accuracy and robustness across multilingual contexts, its deployment in production environments—particularly on consumer-grade Windows hardware—has been constrained by significant resource overheads. The dependency on heavy Python runtimes, the Global Interpreter Lock (GIL), and suboptimal memory management in standard deep learning frameworks have necessitated a shift towards native, compiled implementations.

whisper.cpp, a high-performance inference engine built upon the ggml tensor library, represents the apex of this shift. By decoupling the model weights from the PyTorch ecosystem and implementing tensor operations in plain C/C++, whisper.cpp enables "zero memory allocation at runtime" and broad hardware compatibility.¹ This report provides an exhaustive technical evaluation of whisper.cpp specifically within the Windows operating system environment, addressing the unique challenges of server-mode deployment, persistent context management, and hardware acceleration via CUDA, DirectML, and OpenVINO.

The analysis is driven by the operational necessity to deploy robust, long-running ASR services that can function offline on standard workstations. It scrutinizes critical stability flaws, such as the documented resource handle leak in the Windows server implementation², and evaluates the comparative efficacy of backend accelerators. Furthermore, it explores the mathematical and operational trade-offs of integer quantization (Q4_0, Q5_0, Q8_0) and the integration of advanced optimization techniques like Flash Attention and memory-mapped (mmap) loading to mitigate latency bottlenecks.

2. Core Architecture and Inference Mechanics

To understand the performance characteristics and limitations of whisper.cpp on Windows, one must first dissect the underlying architecture of the inference engine and how it diverges from the reference implementation.

2.1 The ggml Tensor Library and Memory Efficiency

The foundational differentiator of whisper.cpp is its reliance on ggml, a tensor library designed for commodity hardware. Unlike frameworks that prioritize training throughput on cluster-scale hardware, ggml prioritizes inference efficiency on edge devices.

2.1.1 Zero Runtime Allocation

A critical feature of the whisper.cpp architecture is its memory management strategy. In standard Python/PyTorch inference, memory is frequently allocated and deallocated for intermediate tensors during the forward pass. This behavior triggers garbage collection cycles and memory fragmentation, leading to non-deterministic latency spikes—a phenomenon unacceptable in real-time audio processing. whisper.cpp pre-calculates the required compute buffer size at model load time. The `whisper_init_state` function determines the exact memory footprint needed for the Key-Value (KV) cache, the compute buffer for the encoder, and the cross-attention layers.³

For instance, loading a base model requires approximately 156 MB for the CPU buffer, with specific allocations for the KV self-attention (16.52 MB) and compute buffers (85.93 MB for encoding).³ This static allocation strategy ensures that once the model is loaded, the memory footprint remains constant, preventing the gradual memory creep often observed in dynamic languages.

2.1.2 Tensor Operations and SIMD

The library implements platform-specific optimizations for tensor operations. On x86 architectures common to Windows servers, whisper.cpp utilizes AVX, AVX2, and FMA intrinsics to accelerate vector math.¹ The system info logs typically confirm the presence of these instruction sets (`AVX = 1 | AVX2 = 1 | FMA = 1`).³ While AVX-512 support is present in newer builds, the core performance gains on standard consumer CPUs (like the Ryzen 5 or Core i7) are derived from efficient utilization of AVX2 for matrix multiplications during the encoder phase.

2.2 The Encoder-Decoder Inference Pipeline

The execution flow of Whisper is bifurcated into two distinct phases: the Encoder and the Decoder. Understanding the latency distribution between these phases is vital for optimization.

2.2.1 The Encoder Phase

The encoder processes the log-mel spectrogram of the input audio. This is a computationally

intensive, non-autoregressive operation. It runs once per 30-second audio chunk.

- **Performance Profile:** On a GTX 1660 SUPER using CUDA, the encode phase for an 11-second sample (jfk.wav) takes approximately **249.38 ms**.⁴
- **CPU Sensitivity:** On a Ryzen 5 3600 CPU, this same operation takes **1377.10 ms**.⁴ This disparity highlights that the encoder is heavily bound by raw FLOPs (floating-point operations per second). The massive parallelism of GPUs is essential here, providing a 5.5x speedup over the CPU.

2.2.2 The Decoder Phase

The decoder generates text tokens autoregressively. It runs repeatedly, generating one token at a time until the end-of-text token is produced or the context is full.

- **Performance Profile:** The decode phase is significantly faster per iteration but sensitive to memory bandwidth and latency. On the same GTX 1660 SUPER, decoding took **39.69 ms**, while the CPU took **9.44 ms**.⁴
- **The CPU Advantage:** Surprisingly, the CPU can be faster than the GPU for the decoding of short sequences. This is due to the kernel launch overhead associated with CUDA. For small batch sizes (beam size = 1), the time taken to move data to the GPU and launch the kernel exceeds the computation time. However, as beam size increases (e.g., standard beam size of 5), the GPU's throughput advantage typically allows it to overtake the CPU.

2.3 Context Window Management

The Transformer architecture of Whisper relies on a fixed context window. The model is trained to process 30-second log-mel spectrograms. Handling audio streams that exceed this duration requires sophisticated window management.

2.3.1 Sliding Window Logic

For continuous transcription, whisper.cpp employs a sliding window approach. The audio is buffered until it fills the context window. After inference, a portion of the audio is discarded, and new audio is appended.

- **Overlap and Disambiguation:** A critical challenge is preserving semantic context across boundaries. As noted in technical discussions, words cannot be disambiguated in isolation.⁵ A reliable implementation typically requires an overlap of several seconds. The "streaming mode" typically uses a sliding window where only the "stable" tokens (those far from the edge of the window) are committed to the final transcript, while the unstable tokens are re-evaluated in the next window pass.⁵

2.3.2 The no-context Flag

The no-context parameter controls whether the decoder uses the previously generated tokens as a prompt for the current segment.

- **Mechanism:** When no-context is false (default), the KV cache of the decoder retains the history of the conversation. This allows the model to "remember" entities and formatting styles.
- **Risk:** In long-running sessions, this context can accumulate errors, leading to hallucination loops where the model repeats a phrase endlessly.
- **Optimization:** Enabling no-context (true) forces the model to treat each segment independently.² While this degrades the global coherence of the transcript (e.g., losing track of a speaker's name mentioned 2 minutes ago), it drastically improves stability for indefinite server uptime by resetting the decoder state for each request.

3. Server Mode Deployment and Windows Stability Analysis

Deploying whisper.cpp as a persistent server (whisper-server.exe) on Windows introduces specific operational challenges distinct from the CLI usage pattern. The primary objective is to amortize the model load time—which can exceed 300ms for base models and several seconds for large models—across thousands of requests.

3.1 The Persistent Inference Session

In server mode, the application loads the model into memory once and keeps it resident. This eliminates the initialization latency (e.g., **334.40 ms** load time vs **856.96 ms** total time for a single run ⁴). For a short 1-second voice command, removing the load time reduces latency by nearly 40%.

However, the server must manage concurrent access. The current implementation of whisper.cpp is largely single-threaded regarding the inference state. While it can use multiple threads for matrix math (n_threads), it processes requests sequentially. Support for batched decoding is available but requires careful configuration to match the specific concurrency patterns of the workload.⁶

3.2 Critical Defect: The Windows Handle Leak

One of the most significant findings from the operational analysis of whisper-server on Windows is a severe resource leak that compromises long-term stability.

3.2.1 Symptoms and Reproducibility

Users have documented a reproducible failure mode where the server stops returning transcription data after processing approximately 6 to 7 consecutive requests.²

- **Behavior:** The server process does not crash. It continues to accept TCP connections and HTTP requests. However, the response body contains empty JSON: {"text": ""}.
- **Metrics:** Monitoring via Task Manager or Resource Monitor reveals a monotonic increase

in the process handle count.

- **Initial State:** ~306 handles.
- **After Request 8:** ~338 handles.²
- **Trend:** The handle count never decreases, even during idle periods.

3.2.2 Root Cause Analysis

This behavior is characteristic of a "handle leak" in the Windows kernel objects. In the context of whisper-server, this likely involves synchronization primitives associated with the backend or socket management.

- **Backend Independence:** The leak persists regardless of whether the backend is CPU-only, Vulkan, or CUDA.² This suggests the flaw lies in the generic server loop code (likely in server.cpp) or the networking abstraction (using httpplib or raw sockets) rather than the inference kernel itself.
- **WDDM Interaction:** On Windows, the interaction with the GPU driver (WDDM) involves the creation of event objects for fence synchronization. If the server fails to properly release these events after a request is processed, the driver eventually refuses to allocate new synchronization objects, causing the GPU submission to fail silently while the CPU code continues to run.

3.2.3 Architectural Mitigation: The Supervisor Pattern

Given that this is a code-level defect in the current release binaries (v1.8.x), systems architects must implement a "Supervisor" or "Sidecar" pattern for production deployment.

1. **Request Counting:** The supervisor proxies all HTTP traffic to the whisper-server.
2. **Threshold Recycling:** A counter tracks the number of requests processed. Upon reaching a safety threshold (e.g., 5 requests), the supervisor marks the backend as "draining."
3. **Process Rotation:** The supervisor spawns a fresh whisper-server instance in the background. Once the new instance reports health (port open), the supervisor kills the old process and routes new traffic to the new instance.
4. **Latency Impact:** This introduces a latency spike (equal to model load time) every 5 requests, but guarantees 100% availability and circumvents the handle exhaustion.

4. Hardware Acceleration Strategies on Windows

The performance of whisper.cpp on Windows is heavily dictated by the choice of hardware acceleration backend. Unlike Linux, where CUDA is the dominant force, Windows offers a diverse ecosystem including DirectML and OpenVINO.

4.1 NVIDIA CUDA (Compute Unified Device Architecture)

CUDA remains the reference standard for high-performance AI inference. whisper.cpp

integrates with CUDA via cuBLAS for efficient matrix multiplication.

4.1.1 Performance Benchmarks

Comparative benchmarks illustrate the dominance of the C++ CUDA implementation over Python equivalents.

- **System:** GTX 1660 SUPER, Ryzen 5 3600.
- **Whisper.cpp (CUDA):** Total time for 11s audio = **856.96 ms**.⁴
- **OpenAI Whisper (PyTorch/CUDA):** Total time = **1314 ms**.⁴
- **Analysis:** The C++ implementation yields a **35% reduction in latency**. This is attributed to the removal of Python interpreter overhead and more efficient memory management.

4.1.2 Flash Attention Integration

whisper.cpp supports Flash Attention, a technique that optimizes the attention mechanism by tiling computations to fit within the GPU's SRAM (L1/Shared Memory), minimizing expensive reads/writes to VRAM (HBM).

- **Implementation:** Enabled via the `-fa` flag or `WHISPER_CUDA_FLASH_ATTENTION` build option.¹
- **VRAM Impact:** Flash Attention significantly reduces the memory footprint of the attention layers. Benchmarks on large models show VRAM usage dropping from **4.7 GB** to **4.1 GB**.⁷ This 600 MB saving is crucial for running "Large" models on 6GB or 8GB cards, preventing Out-Of-Memory (OOM) errors and allowing for larger batch sizes.

4.2 Microsoft DirectML (Direct Machine Learning)

DirectML is Microsoft's low-level API for hardware-accelerated machine learning, capable of targeting any GPU compatible with DirectX 12. This includes NVIDIA GeForce, AMD Radeon, and Intel Arc.

4.2.1 The "WhisperDesktop" Implementation

A specific implementation of Whisper using DirectCompute (a closely related technology to DirectML) known as "WhisperDesktop" has demonstrated superior performance characteristics on Windows, challenging the CUDA hegemony.

- **Throughput:** On an RTX 4090, WhisperDesktop achieved over **3,000 Words Per Minute (WPM)**. In contrast, the standard PyTorch implementation reached only ~1,600 WPM on the same hardware.⁸
- **Memory Efficiency:** The DirectML/DirectCompute approach is vastly more efficient. It allows the **Large** model to run on GPUs with only **6 GB VRAM**. The PyTorch version typically requires **10 GB**, locking out users with RTX 3060 (6GB) or RTX 2060 cards.⁸
- **CPU Bottleneck Elimination:** The PyTorch implementation is heavily CPU-bound on Windows. Using a slower CPU (i9-9900K vs i9-13900K) doubled transcription time in

PyTorch, whereas the C++ DirectML implementation remained GPU-saturated.⁸

This data suggests that for Windows deployments targeting a broad range of consumer hardware, a DirectML-based backend is the optimal strategy. It provides vendor agnosticism and superior VRAM efficiency.

4.3 Intel OpenVINO

For deployments on standard business laptops lacking discrete GPUs, Intel's OpenVINO toolkit provides a critical acceleration path for integrated graphics (Iris Xe) and CPUs.

- **CPU Optimization:** OpenVINO leverages AVX-512 and VNNI instructions more effectively than generic OpenBLAS builds. Benchmarks show that enabling BLAS on CPU improves speed by 3-4x.⁶
- **Integration:** whisper.cpp supports OpenVINO via a specific build flag. While it adds a dependency on the OpenVINO runtime DLLs, it transforms a standard Ultrabook into a viable transcription server, capable of real-time performance on "Base" or "Small" models.

5. The Mathematics and Efficacy of Quantization

To enable large models on consumer hardware, whisper.cpp utilizes aggressive integer quantization. This process maps the 16-bit floating-point weights of the neural network into lower-precision integers (4-bit, 5-bit, or 8-bit), reducing both model size and memory bandwidth usage.

5.1 Quantization Methodologies

5.1.1 Block-wise Quantization

The ggml library employs block-wise quantization. Weights are grouped into blocks (typically 32 or 64 values). Each block stores a high-precision scaling factor (float16) and the low-precision quantized weights.

- **Reconstruction:** $w_{approx} = s_{block} \times q_{weight}$
This hybrid approach preserves the dynamic range of the weights better than uniform quantization across the entire tensor.

5.2 Comparative Analysis of Modes

5.2.1 Q4_0 (4-bit)

- **Compression:** High. Reduces model size by ~75% vs FP16.
- **Performance:** Fastest memory throughput.
- **Quality:** Highest degradation. Perplexity increases significantly.⁹ In ASR terms, this manifests as increased Word Error Rate (WER), particularly with homophones or

mumbled speech where the model relies on subtle acoustic cues that 4-bit precision may obliterate.

5.2.2 Q5_0 (5-bit)

- **The "Sweet Spot":** Q5_0 uses 5 bits per weight plus the scaling factor.
- **Quality:** It offers a perplexity virtually indistinguishable from 8-bit or FP16 models in many benchmarks, while still providing substantial size reduction.⁹
- **Recommendation:** This is often the recommended default for consumer deployment where VRAM is tight (e.g., fitting a Large model on 4GB-6GB VRAM).

5.2.3 Q8_0 (8-bit)

- **Fidelity:** Near-lossless. The perplexity is almost identical to the uncompressed FP16 model.⁹
- **Bandwidth:** Reduces memory bandwidth usage by 50% compared to FP16.
- **Deployment:** Ideal for production servers where accuracy is paramount but bandwidth efficiency is still required.

5.3 The "Turbo" Variant

Recent updates have introduced "Turbo" variants of the Whisper models (e.g., large-v3-turbo).

- **Architecture:** These models typically reduce the number of decoder layers while keeping the encoder robust.
- **Performance:** The large-v3-turbo-q8_0 model (873 MB) processes audio in **14 seconds**, compared to **52 seconds** for the standard large-v3.¹⁰
- **Trade-off:** The reduced decoder depth can impact the transcription of complex, overlapping speech or "verbatim" tasks like lyrics transcription, where it may miss background vocals. However, for standard dialogue, the speedup is transformative.

6. Resource Monitoring on Windows: The WDDM Challenge

Effective server management requires precise monitoring of GPU usage. On Windows, this is complicated by the WDDM architecture, which virtualizes GPU memory.

6.1 Limitations of Standard Tools

Standard NVIDIA tools like nvidia-smi often fail to report per-process VRAM usage on Windows.

- **WDDM vs TCC:** In TCC (Tesla Compute Cluster) mode, the GPU is dedicated to compute, and strict accounting is available. In WDDM mode (used for display output), the OS manages memory paging. nvidia-smi frequently reports "N/A" for processes running in

WDDM mode.¹¹

6.2 Advanced Monitoring APIs

6.2.1 Performance Counters

The most reliable high-level metric is provided by Windows Performance Counters.

- **Counter:** \GPU Engine(*)\Utilization Percentage.¹¹
- **Granularity:** This allows monitoring the 3D, Compute, and Copy engines separately. However, mapping these engines to a specific process ID (PID) requires correlating the "instance name" of the counter with the process list, which is brittle in multi-instance environments.

6.2.2 The Undocumented D3DKMT API

For precise, per-process GPU usage, developers must utilize the undocumented D3DKMTQueryStatistics API.¹²

- **Mechanism:** This kernel-mode API allows an application to query the GPU scheduler directly. By iterating through the "nodes" (hardware engines) associated with a process handle, a supervisor can sum the running time of the threads.
- **Implementation:** Accessing this requires ctypes in Python or C++. It involves loading gdi32.dll or d3dkmthk.dll, opening the adapter LUID, and interpreting the D3DKMT_QUERYSTATISTICS C-structure. This provides the exact GPU time consumed, bypassing the virtualization abstractions of WDDM.

6.3 Detecting Game Mode

For client-side "assistant" deployments, it is crucial to detect if the user is in a full-screen game to pause inference and release resources.

- **API:** SHQueryUserNotificationState. A return value of QUNS_RUNNING_D3D_FULL_SCREEN indicates an exclusive-mode game is running.¹³
- **Registry:** Checking HKEY_CURRENT_USER\Software\Valve\Steam\RunningAppID reliably detects active Steam games.¹³

7. Python Bindings and Integration Strategies

While whisper.cpp is a C++ library, Python bindings are essential for integration into the broader ML ecosystem.

7.1 Binding Comparison

7.1.1 pywhispercpp

- **Design:** A fairly direct wrapper around the C++ API.

- **Persistence:** Supports persistent model loading via the Model class. The model is loaded once, and the transcribe method can be called repeatedly on the instance.¹⁴
- **Streaming:** Includes examples for streaming and livestreaming, utilizing parameters like `single_segment=True` to handle continuous audio flow.¹⁴

7.1.2 whisper-cpp-python

- **Focus:** Often aims for API compatibility with the original OpenAI Whisper Python package.
- **Performance:** Generally inherits the performance benefits of the underlying C++ library but adds a thin layer of Python object overhead.

7.2 Model Persistence Implementation

To implement persistent context in Python using `pywhispercpp`:

Python

```
from pywhispercpp.model import Model

# Load model once (approx 300-500ms)
model = Model('base.en', n_threads=6, print_realtime=False)

# Loop for requests
while True:
    audio_segment = get_next_audio_chunk()
    # Inference (approx 40-80ms for 1s audio)
    segments = model.transcribe(audio_segment, new_segment=True)
    print(segments.text)
```

This pattern ensures that the initialization cost is paid only once. The Model object maintains the internal `whisper_context` pointer.

8. Optimization Techniques: Loading and Latency

8.1 Memory Mapping (mmap)

`whisper.cpp` utilizes system-level memory mapping (`mmap`) to load models.

- **Mechanism:** Instead of reading the file into a buffer, the OS maps the file's disk blocks to the process's virtual address space.
- **Latency:** This makes "loading" the model nearly instantaneous, as no physical read

occurs until the memory is accessed.

- **System Health:** The OS can discard unmodified pages of the model from RAM if the system is under memory pressure, reloading them transparently from disk when needed. This is superior to malloc, which locks physical RAM.
- **Build Flag:** Typically enabled by default, but can be controlled via build flags. Disabling mmap (`--no-mmap`) forces a full read, which may improve performance slightly on slow HDDs by pre-caching everything, but degrades startup time.¹⁵

8.2 CPU vs GPU Crossover

A critical architectural decision is determining when to offload to GPU.

- **Small Models:** For tiny and base models, the CPU (with AVX2) is often sufficient. The overhead of PCI-E data transfer can make GPU inference slower for very short audio clips (< 2 seconds).
- **Large Models:** For medium and large, the GPU is mandatory. The computational density of the Transformer layers scales quadratically, and CPU inference becomes perceptibly slow (RTF > 1), breaking real-time guarantees.¹⁷

9. Conclusion and Strategic Recommendations

The deployment of whisper.cpp on Windows servers offers a high-performance, cost-effective alternative to cloud-based ASR APIs and heavy Python-based local deployments. By leveraging the ggml library's efficient memory management and backend flexibility, developers can achieve throughputs exceeding **3,000 WPM** on consumer hardware.

However, operational stability requires rigorous architectural mitigation. The **Windows handle leak** in whisper-server mandates the use of a supervisor process to recycle instances periodically. Performance optimization relies on a strategic selection of backends: **DirectML** for broad compatibility and memory efficiency, and **CUDA** with **Flash Attention** for maximum raw throughput on NVIDIA silicon.

Strategic Recommendations:

1. **Backend:** Adopt **DirectML** as the primary backend for general Windows distribution to maximize hardware support and VRAM efficiency. Use **CUDA** only in controlled environments with NVIDIA GPUs.
2. **Quantization:** Standardize on **Q5_0** or **Q8_0** quantization. Q5_0 offers the best balance of VRAM saving and fidelity for "Large" models.
3. **Architecture:** Implement a **"Sidecar" Supervisor** to monitor the whisper-server process, strictly cycling it every 10-20 requests to preempt handle exhaustion.
4. **Monitoring:** Integrate **Game Mode detection** and **Performance Counter** monitoring to ensure the background service yields resources to foreground applications, maintaining a seamless user experience.

By adhering to these protocols, whisper.cpp can serve as the backbone of a robust, privacy-centric, and highly responsive speech recognition infrastructure on the Windows platform.

10. References

¹ GitHub: whisper.cpp repository features and platforms. ⁵ JUCE Forum: Streaming mode and sliding window context. ² GitHub Issue #3358: Resource leak in whisper-server on Windows. ⁴ Benchmark: whisper.cpp CUDA vs CPU vs OpenAI PyTorch. ¹¹ StackOverflow: NVML limitations on Windows WDDM. ² GitHub Issue: no-context flag and persistent session management. ¹⁰ Quantization benchmarks: Q8_0 Turbo vs Large-v3. ¹⁴ pywhispercpp documentation: Model persistence. ¹³ SuperUser: Windows Game Mode detection APIs. ⁸ Tom's Hardware: WhisperDesktop DirectML vs PyTorch CUDA benchmarks. ⁶ GitHub Issue #1127: CPU vs GPU performance comparison. ⁹ Arxiv: Quantization trade-offs (Q4 vs Q5 vs Q8). ⁷ Faster-Whisper: Flash Attention VRAM usage. ³ GitHub Discussion: Memory usage logs for base model. ¹² StackOverflow: D3DKMTQueryStatistics API usage. ¹ Whisper.cpp: Flash Attention build flags.

Works cited

1. ggml-org/whisper.cpp: Port of OpenAI's Whisper model in C/C++ - GitHub, accessed February 10, 2026, <https://github.com/ggml-org/whisper.cpp>
2. [BUG] Handle leak in whisper-server on Windows causes empty ..., accessed February 10, 2026, <https://github.com/ggml-org/whisper.cpp/issues/3358>
3. Perf comparison with OpenAI whisper model vs cpp model (Windows) #1527 - GitHub, accessed February 10, 2026, <https://github.com/ggerganov/whisper.cpp/discussions/1527>
4. Perf comparison with OpenAI whisper model vs cpp model ... - GitHub, accessed February 10, 2026, <https://github.com/ggml-org/whisper.cpp/discussions/1527>
5. OpenAI Whisper C++ and JUCE, accessed February 10, 2026, <https://forum.juce.com/t/openai-whisper-c-and-juce/61230>
6. Comparison with faster-whisper · Issue #1127 · ggml-org/whisper.cpp, accessed February 10, 2026, <https://github.com/ggerganov/whisper.cpp/issues/1127>
7. Faster Whisper transcription with CTranslate2 - GitHub, accessed February 10, 2026, <https://github.com/SYSTRAN/faster-whisper>
8. OpenAI Whisper Audio Transcription Benchmarked on 18 GPUs - Tom's Hardware, accessed February 10, 2026, <https://www.tomshardware.com/news/whisper-audio-transcription-gpus-benchmarked>
9. Which Quantization Should I Use? A Unified Evaluation of Llama.cpp Quantization on Llama-3.1-8B-Instruct - arXiv, accessed February 10, 2026, <https://arxiv.org/html/2601.14277v1>
10. Blog article: Comparing Whisper Models for Transcribing Queen's ..., accessed February 10, 2026, <https://github.com/ggml-org/whisper.cpp/discussions/3074>

11. How to measure GPU usage per process in Windows using python? - Stack Overflow, accessed February 10, 2026,
<https://stackoverflow.com/questions/51702335/how-to-measure-gpu-usage-per-process-in-windows-using-python>
12. How do I get GPU usage per process? - Stack Overflow, accessed February 10, 2026,
<https://stackoverflow.com/questions/13017148/how-do-i-get-gpu-usage-per-process>
13. How to check If Windows game mode is currently active? [closed] - Super User, accessed February 10, 2026,
<https://superuser.com/questions/1829954/how-to-check-if-windows-game-mode-is-currently-active>
14. PyWhisperCpp, accessed February 10, 2026,
<https://absadiki.github.io/pywhispercpp/>
15. Weekly GitHub Report for Llama.cpp: January 20, 2025 - Buttondown, accessed February 10, 2026,
<https://buttondown.com/weekly-project-news/archive/weekly-github-report-for-llamacpp-january-20-2025/>
16. mmap() vs. reading blocks - c++ - Stack Overflow, accessed February 10, 2026,
<https://stackoverflow.com/questions/45972/mmap-vs-reading-blocks>
17. Optimizing Whisper Speed: CPU vs. AMD GPU? : r/LocalLLaMA - Reddit, accessed February 10, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1gsvyyh/optimizing_whisper_speed_cpu_vs_amd_gpu/